

Politechnika Warszawska

WYDZIAŁ MECHANICZNY
ENERGETYKI I LOTNICTWA



Instytut Techniki Lotniczej i Mechaniki Stosowanej

Praca przejściowa magisterska

na kierunku Lotnictwo i Kosmonautyka
w specjalności Kosmonautyka

Stanowisko pomiarowe odpowiedzi układu wykonawczego systemu
sterowania rakiet do badań Hardware-in-the-Loop

inż. Olgierd Skromak

Numer albumu 320987

promotor

dr inż. Dariusz Miedziński

WARSZAWA 2026

Spis treści

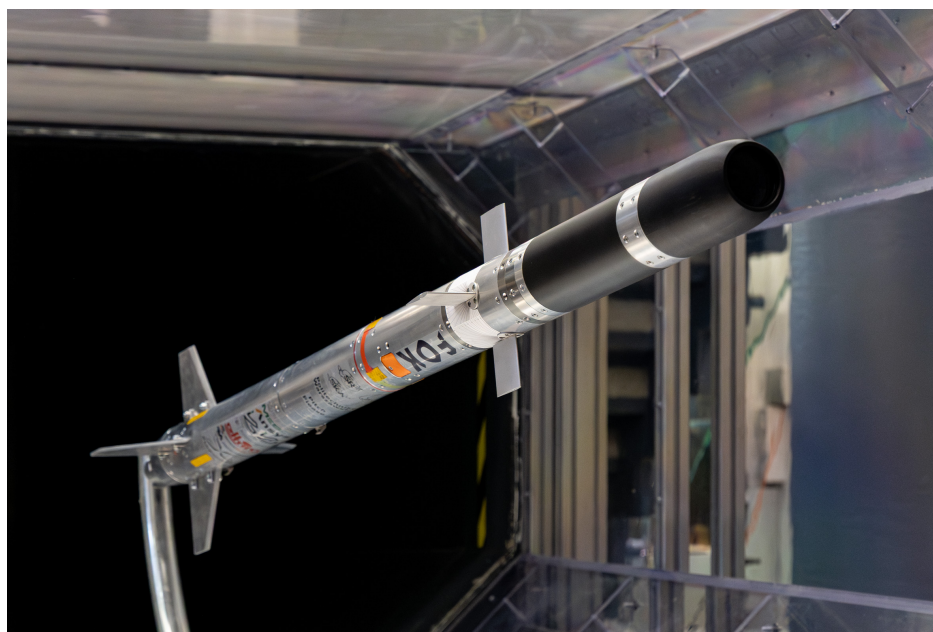
1. Wstęp	5
2. Badany obiekt	6
2.1. System sterowania rakiety	7
2.1.1. Komputer pokładowy	7
2.1.2. Układ wykonawczy systemu sterowania	8
2.1.3. Jednostka pomiarowa INS/GNSS	8
3. Motywacja i cel pracy	9
3.1. Cele pracy	9
4. Casket	11
4.1. Założenia	11
4.2. Projekt i realizacja	12
4.2.1. Symulacja obciążenia aerodynamicznego	14
4.3. Oprogramowanie	15
4.4. Użytkowanie	16
4.5. Przygotowanie stanowiska	16
4.6. Umieszczenie rakiety w stanowisku	16
4.7. Przeprowadzenie testu	17
5. Symulacja HIL	19
5.1. Dostosowanie symulacji do badań HIL	19
5.1.1. Komputer osobisty	20
5.1.2. Komputer czasu rzeczywistego	21
6. Badania	26
6.1. Identyfikacja modelu	26
6.2. Symulacja HIL	28
7. Rozwój	31
7.1. Wbudowana w <i>CASKET</i> korekta wychyleń sterów	31
7.2. Zwiększenie częstotliwości <i>CASKETa</i>	31
8. Podsumowanie	33
Bibliografia	35
Wykaz symboli i skrótów	37
Skróty	37
Symbole	37
Spis rysunków	38
Spis tabel	38

1. Wstęp

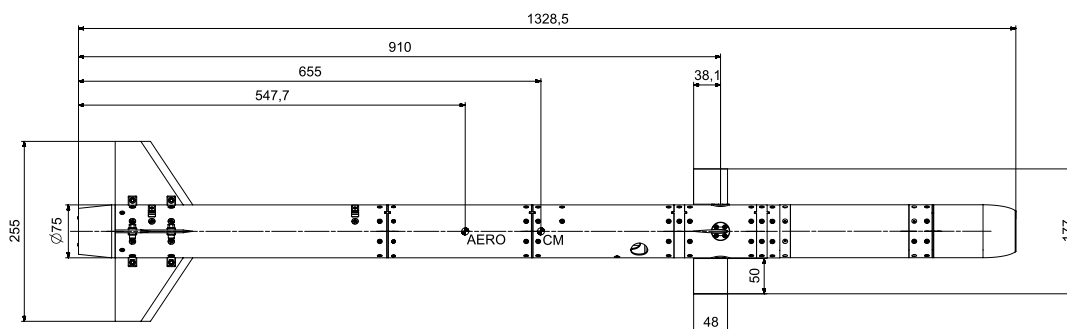
Praca dotyczy przeprowadzania symulacji Hardware-in-the-Loop (HIL) rakiety sterowanej FOK 2 rozwijanej w Studenckim Kole Astronautycznym. W szczególności praca dotyczy modelowania i przeprowadzania symulacji HIL układu wykonawczego systemu sterowania rakiety (CAS). Stworzone stanowisko badawcze powstało w ramach prac nad artykułem [1] oraz niniejszej pracy.

2. Badany obiekt

Rakieta FOK 2 jest wyposażoną w głowicę obrazującą ewolucję rakiety sterowanej FOK. Rakieta, przedstawiona na Rysunku 2.1, sterowana jest aerodynamicznie za pomocą 4 sterów umieszczonych przed środkiem masy (canardów). Napędzana jest silnikiem na stały materiał pędny o impulsie około 1000 Ns. Pozwala to jej osiągnąć pułap 1000 m oraz liczbę Macha 0.4000. Parametry rakiety FOK 2 przedstawiono w Tabeli 2.1. Wymiary rakiety przedstawiono na Rysunku 2.2. Obecnie rozwijany jest nowy zespół napędowy - BEM [2]. Wraz z odchudzeniem reszty rakiety pozwoli on na znaczny wzrost osiągnięć rakiety - osiągnięcie liczby Macha 0.7000 i pułapu 3.000 km. Celem rakiety FOK 2 jest lot do celu z użyciem nawigacji wizyjnej. Na dzień stworzenia pracy rakieta ta odbyła 5 testów lotnych podczas których zbierane były dane do identyfikacji modelu aerodynamicznego rakiety oraz podjęte były próby pionizacji rakiety. Niestety żadna z podjętych misji pionizacyjnych nie zakończyła się powodzeniem.



Rysunek 2.1. Rakieta FOK 2 w tunelu aerodynamicznym



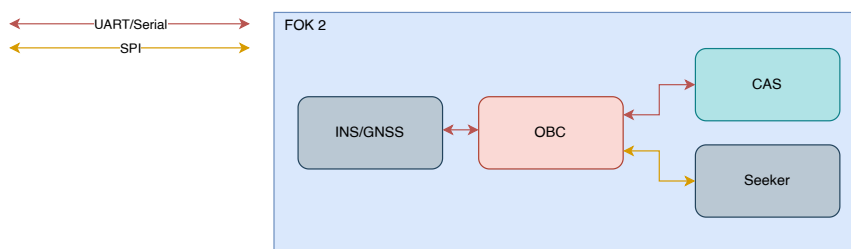
Rysunek 2.2. Wymiary rakiety FOK 2

Tabela 2.1. Parametry rakiety FOK 2.

Parametr	Wartość	Jednostka
Masa startowa	6,0	kg
Długość	1,3285	m
Średnica	0,075	m
Impuls całkowity	1000	N s
Masa materiału pędnego	0,5	kg
Pułap	1000	m

2.1. System sterowania rakiety

Docelowa pętla sterowania (Rysunek 2.3) rakiety FOK 2 opiera się na interakcji między głowicą optoelektroniczną, czujnikiem INS/GNSS, komputerem pokładowym i układem wykonawczym systemu sterowania. W tej pracy rozważane są konfiguracje rakiety bez informacji z głowicy w pętli sterowania.

**Rysunek 2.3.** Pętla sterowania rakiety FOK 2

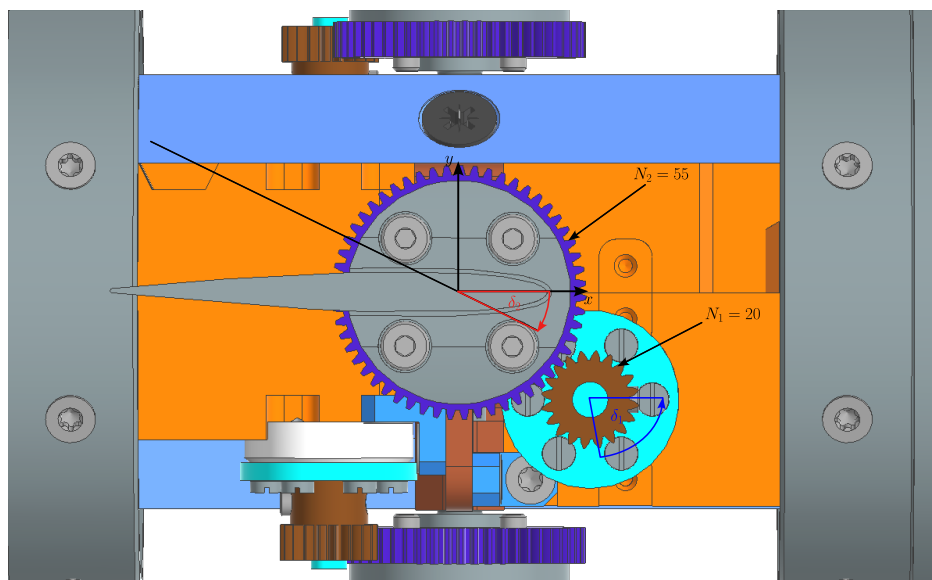
2.1.1. Komputer pokładowy

Komputer pokładowy rakiety FOK 2 opiera się na Raspberry Pi Pico 2 W. Został on wybrany przez łatwość pisania oprogramowania na tej platformie. Oprogramowanie komputera pokładowego jest autorskie, oparte na systemie czasu rzeczywistego FreeRTOS [3]. Podczas lotu zbiera on informacje za pomocą protokołu UART z czujnika. Także z użyciem tego protokołu komunikuje się z serwomechanizmami. Z głowicą komunikuje się za pomocą protokołu SPI, druga instancja tego protokołu jaką dysponuje Pico jest przeznaczona na obsługę pamięci flash zapisującej stan misji. Przewodowe możliwości komunikacji z mikrokontrolerem uzupełnia port USB, po którym jest udostępniona konsola umożliwiająca konfigurację i komunikację z komputerem pokładowym. Ponad wymienione interfejsy jest jeszcze WiFi, które podczas testu lotnego służy do nadawania telemetry przez komputer pokładowy gdy rakieta znajduje się na wyrzutni. Interfejs ten zyskuje na znaczeniu podczas testów HIL przeprowadzanych na komputerze osobistym, podczas których służy do przekazywania stanu rakiety z symulacji do komputera pokładowego rakiety i pożądanego wychylenia sterów do symulacji. Podczas testów HIL przeprowadzanych na komputerze czasu rzeczywistego może służyć do transmisji stanu komputera pokładowego w celu jego zapisu do dalszej analizy.

2.1.2. Układ wykonawczy systemu sterowania

Układ wykonawczy badanej w pracy konfiguracji rakiety FOK 2 (FOK 2D) składa się z:

- serwomechanizmów HerkuleX DRS 0101,
- redukującej przekładni zębatej o przełożeniu $i_w = \frac{55}{20} = 2,75$,
- steru o profilu NACA 0009 i wymiarach przedstawionych na Rysunku 2.2,
- elementów struktury mechanicznej i oprzyrządowania elektronicznego.



Rysunek 2.4. Układ wykonawczy systemu sterowania rakiety FOK 2

W nowszych wersjach rakiety (od FOK 2E) używany jest układ wykonawczy nieoparty o koła zębate, a o popychacze. Wprowadza to drobne nieliniowości w przełożeniu oraz prowadzi do konieczności ponownego przeprowadzenia części testów identyfikujących parametry układu wykonawczego. Zmiana wynikała głównie z chęci eliminacji luzów występujących w systemie sterowania rakiety.

2.1.3. Jednostka pomiarowa INS/GNSS

W rakiecie używana jest jednostka pomiarowa INS/GNSS VectorNav 200 [4]. Pozwala ona na wyznaczenie parametrów lotu rakiety. Komunikacja między raketą a czujnikiem odbywa się za pomocą protokołu UART. Został stworzony i umieszczony w symulacji HIL symulator pakietów czujnika, który pozwala na bezpośrednie wpięcie symulacji w pętlę sterowania rakiety. Model czujnika używany w symulacji jest minimalny, pozbawiony symulacji niepewności i szumów. Planowane jest zaimplementowanie dokładniejszego modelu czujnika, który pozwoli na weryfikację działania algorytmu sterowania rakiety w warunkach bardziej zbliżonych do rzeczywistych.

3. Motywacja i cel pracy

Stworzenie dokładnego modelu symulacyjnego rakiety sterowanej FOK 2 jest krokiem niezbędnym do powodzenia misji rakiety - dolecenia do celu. Wymaga to stworzenia modeli ruchu, aerodynamiki, oprogramowania, a także modelu układu wykonawczego systemu sterowania rakiety. Stworzenie modelu układu wykonawczego wymaga zidentyfikowania modelu, który będzie w stanie odwzorować zachowanie rzeczywistego układu wykonawczego oraz jego parametrów. Zachowanie całego systemu różni się od zachowania jego poszczególnych elementów, dlatego taki model należy tworzyć w oparciu o badania całego układu. Ponadto w [5] zidentyfikowano potrzebę wprowadzenia poprawek do zadanych wychyleń sterów w celu mitygacji błędów enkoderów poszczególnych serwomechanizmów. Została więc zidentyfikowana potrzeba stworzenia stanowiska badawczego, które umożliwi powtarzalny pomiar odpowiedzi układu wykonawczego systemu sterowania rakiety w celu zebrania danych do identyfikacji modelu. Jednym z etapów testowania rakiet jest przeprowadzanie symulacji Hardware-in-the-Loop (HIL). Ma on na celu weryfikację modeli symulacyjnych, oprogramowania i integracji systemów. Takie testy są tym bliższe rzeczywistemu lotowi im bardziej go odzwierciedlają.

O ile w przypadku rakiety FOK 2 możliwe jest wpięcie bezpośrednio komputera pokładowego rakiety do symulacji, to wymaga to wprowadzenia zmian w konfiguracji oprogramowania względem konfiguracji lotnej. Może to powodować różnice w zachowaniu systemu a tym samym zmniejszać wierność symulacji. Dodanie do symulacji HIL rzeczywistego układu wykonawczego systemu sterowania pozwala na urzeczywistnienie wyjścia z algorytmu sterowania - wychylenia sterów. Wprowadzenie zewnętrznego urządzenia pomiarowego pozwala, natomiast na uniezależnienie się od ograniczeń wbudowanego w ракетę systemu pomiarowego wychylenia sterów, który jest obciążony błędami i ograniczeniami. Ponadto, pozwala to na zmiętygowanie opóźnień związanych z przetwarzaniem i transmisją sygnału.

Ostatecznym celem zespołu rozwijającego ракетę FOK 2 w obszarze możliwości przeprowadzania symulacji HIL jest stworzenie systemu umożliwiającego na przeprowadzenie misji rakiety lotu do celu całkowicie w symulacji HIL, bez potrzeby przeprowadzania lotu testowego. Podczas takiej symulacji raketa nie powinna *widzieć* różnicy między nią, a rzeczywistym lotem. Do przeprowadzenia takiej symulacji do omawianego w tej pracy stanowiska, wymagane jest dołożenie stanowiska testowego dla systemu wizyjnego (SPACE) rozwijanego w ramach [6] tworząc tym samym kompletne stanowisko testów HIL rakiety FOK 2 (SHED) [7].

3.1. Cele pracy

Praca ma stanowić przede wszystkim charakter użytkowy w prowadzeniu badań HIL rakiety FOK 2, a w szczególności jej układu wykonawczego. Postawiono więc następujące cele pracy:

- Stworzenie dokumentacji stanowiska badawczego układu wykonawczego systemu sterowania rakiety FOK 2 (CASKET),

3. Motywacja i cel pracy

- Udokumentowanie symulacji HIL zarówno na komputerze osobistym jak i komputerze czasu rzeczywistego, Umożliwienie przeprowadzenia badań HIL rakiety FOK 2 członkom Zespołu.
- Identyfikacja modelu układu wykonawczego systemu sterowania rakiety FOK 2.
- Przedstawienie ograniczeń, problemów i pól do dalszych ulepszeń stworzonego stanowiska badawczego.
- Przedstawienie dalszych kierunków badań HIL rakiety FOK 2.

4. Casket

CASKET jest stanowiskiem do pomiaru wychylenia sterów raket. Jego zastosowania to: identyfikacja parametrów systemów sterowania oraz przeprowadzanie symulacji HIL.

4.1. Założenia

Dokładne modelowanie układu wykonawczego systemu sterowania rakiety (ang. Control Actuation System, CAS) jest kluczowe podczas projektowania algorytmów sterowania. FOK 2 jest w stanie mierzyć wychylenie sterów, a konkretnie kąt obrotu serwomechanizmów. Jednak, w [5], zauważono, że serwomechanizmy mierzą kąt obrotu ze znacznym błędem. Różnica między faktycznym wychyleniem steru, a odczytem z serwomechanizmu jest ponadto zwiększona przez luzy obecne w systemie sterowania. W związku z tym, nie jest możliwy dokładny pomiar wychylenia steru, przez wbudowany w raketę serwomechanizm. Chęć przeprowadzania symulacji HIL przedstawia przed wbudowanym układem pomiarowym dodatkowe wyzwania. Są to: opóźnienie, częstotliwość i dokładność pomiaru. Przez ograniczenia w częstotliwości zapytań do serwomechanizmów możliwe jest ich odpypywanie z częstotliwością 50.00 Hz. Wprowadza to dodatkowe, niedopuszczalne opóźnienia do symulacji HIL. Dokładność pomiaru kąta obrotu tarczy serwomechanizmu jest ograniczona przez jego rozdzielczość, która wynosi 0.3250° . Daje to dokładność pomiaru wychylenia steru na poziomie 0.1250° . Mniejsze różnice w obrocie nie są w ogóle mierzone czyniąc pomiar szybkości wychylenia steru niepraktycznym.

Celem stworzenia stanowiska CASKET jest stawienie czoła przed tymi problemami. Stanowisko zostało zaprojektowane z następującymi założeniami [1]:

1. umożliwienie pomiaru wychylenia sterów, nie serwomechanizmów,
2. pomiar wychylenia sterów z większą dokładnością niż wbudowany w raketę system.
3. możliwość pomiaru szybkości wychylenia sterów,
4. pomiar wychylenia sterów z dużą częstotliwością, cel został ustalony na częstotliwość symulacji - 1600 Hz,
5. komunikacja zarówno z komputerem osobistym, jak i z komputerem czasu rzeczywistego - Speedgoat,
6. możliwość charakteryzacji następujących parametrów układu wykonawczego
 - nieliniowości w układzie pomiarowym,
 - luzy w układzie wykonawczym,
7. ocena osiągnięć systemu podczas pracy z obciążeniem.

Ponadto kilka celów funkcjonalnych zostało ustalonych w celu zwiększenia użyteczności stanowiska:

- system powinien być łatwy w obsłudze, tak by możliwe było jego użytkowanie z minimalnym przeszkoleniem,
- system powinien być dostosowywalny do różnych średnic raket i wielkości sterów.
- system powinien się nadawać do przeprowadzania testów przed testem lotnym rakiety.

Ostatni cel sugeruje, że używanie systemu nie powinno wymagać wprowadzania zmian w rakiecie względem konfiguracji lotnej. Sugeruje też, że umieszczenie rakiety w stanowisku powinno być szybkie.

4.2. Projekt i realizacja

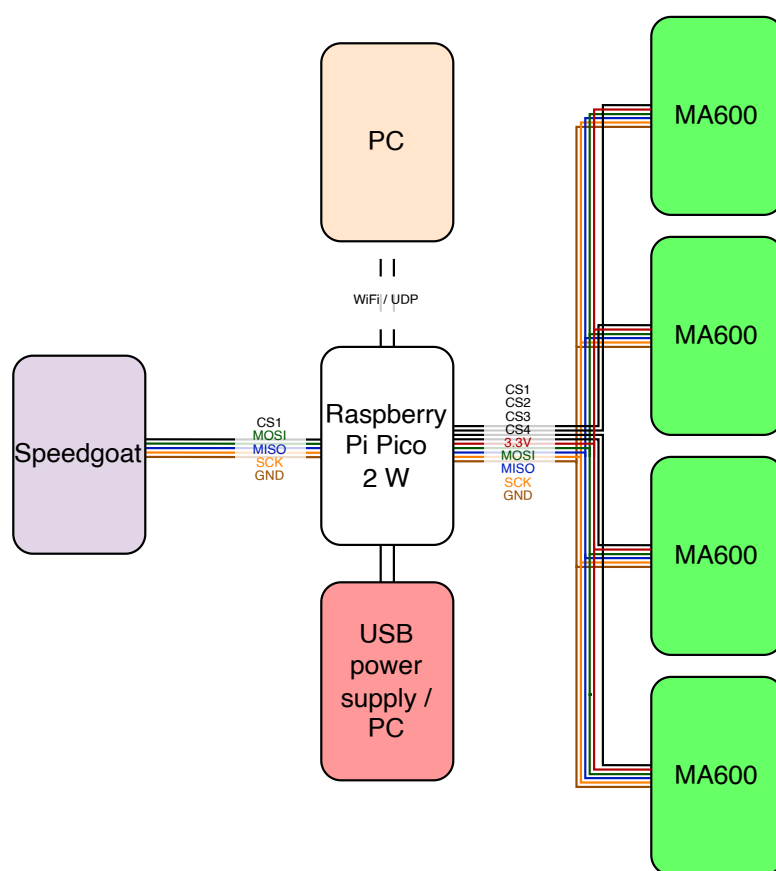
Rdzeniem zaprojektowanego stanowiska jest układ pomiarowy wychylenia sterów. Do pomiaru wychylenia sterów użyto enkodera magnetycznego MA600 [8]. Enkoder magnetyczny został wybrany ze względu na ułatwienie montażu i demontażu rakiety w stanowisku - eliminuje on konieczność mocowania CAS w CASKETcie co pozwala na szybsze umieszczenie rakiety w stanowisku. Ten konkretny enkoder został wybrany ze względu na jego dokładność, ciągły pomiar kąta obrotu, szybkość odczytu, pomiar prędkości obrotu, duże pasmo przenoszenia, możliwości konfiguracji a także interfejs SPI. Spodziewane komplikacje wynikające z użycia enkodera magnetycznego to błędy wynikające ze złego ustawienia magnesu względem enkodera oraz kalibracja.

4 sensory są połączone z mikrokontrolerem Raspberry Pi Pico 2 W, który jest odpowiedzialny za odczyt danych z enkoderów i ich przesyłanie do komputera osobistego lub komputera czasu rzeczywistego. Głównym interfejsem używanym do komunikacji z komputerem czasu rzeczywistego jest SPI. Dokonano prób użycia tego interfejsu do komunikacji z komputerem osobistym jednak nie udało się znaleźć rozwiązania, które byłoby zarówno proste w konfiguracji, użyciu i działałoby z *Simulinkiem*. Wypróbowano użycie konwertera sygnału, jednak nie znaleziono rozwiązania dającego satysfakcjonujące rezultaty. Obecnie do komunikacji z komputerem osobistym używany jest port szeregowy dostępny przez USB lub WiFi. Schemat połączeń jest przedstawiony na Rysunku 4.1.

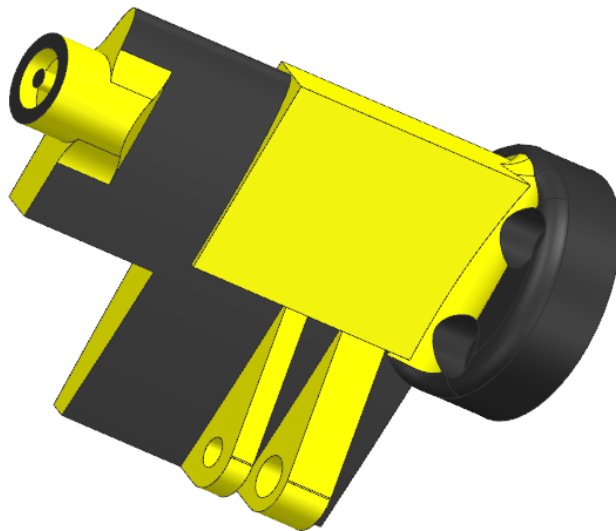
Czujniki zamocowane są na platformach, których pozycja w płaszczyźnie poziomej jest regulowana za pomocą stolików XY wyposażonych w śruby mikrometryczne. Pozwala to na kompensację ustawienia sensora względem magnesu.

Magnesy są umieszczone na specjalnie zaprojektowanych zamiennikach sterów rakiety, w ich osi obrotu. Zamienniki sterów są ponadto wyposażone w zaczepy umożliwiające zamocowanie ich do sprężyn, które symulują obciążenie aerodynamiczne. Zamienniki sterów są przedstawione na Rysunku 4.2.

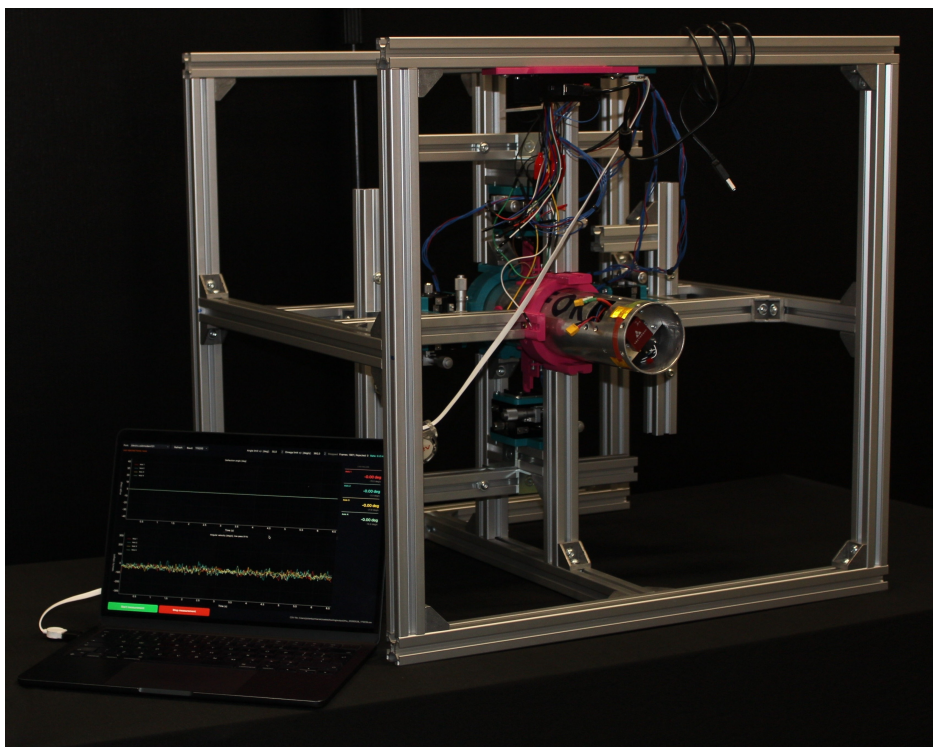
Stanowisko jest wykonane z profili aluminiowych, ma wymiary 0.5600 m x 0.6100 m x 0.6100 m. Rakieta jest mocowana w stanowisku za pomocą adapterów wykonanych technologią druku 3D. Zastosowanie adapterów pozwala na dostosowanie stanowiska do rakiet o różnych średnicach. Bez zmian w użytych profilach aluminiowych możliwe jest testowanie rakiet o średnicach do 110.0 mm. Maksymalna odległość między osią rakiety a końcem symulatora steru może wynieść 157.0 mm. Obecnie stanowisko przystosowane jest do testowania rakiety FOK 2. CASKET przedstawiony jest na Rysunku 4.3 .



Rysunek 4.1. Casket connectivity diagram



Rysunek 4.2. Zamienniki sterów rakiety FOK 2 z magnesami i zaczepami do sprężyn



Rysunek 4.3. FOK 2 w stanowisku CASKET [1]

4.2.1. Symulacja obciążenia aerodynamicznego

W ramach obliczeń przeprowadzonych w ramach [9] wyznaczono moment zawiasowy steru dla wychylenia steru $\delta_{\max} = 10.00^\circ$ przy liczbie Macha $Ma = 0.3000$ na poziomie morza. Wartość ta wynosi $0.02800 \text{ N}\cdot\text{m}$. Celem jest oszacowanie maksymalnej wartości

momentu zawiasowego steru z góry. Rakieta FOK obecnie osiąga prędkość maksymalną na poziomie $Ma = 0.4$, jednak jest planowane zwiększenie osiągniętej przez nią liczby Macha do $Ma = 0.7$. W związku z tym, po przeskalowaniu wartości momentu zawiasowego steru dla $Ma = 0.7$ otrzymano wartość 0.1524 N m . Serwomechanizm HerkuleX DRS-0101 użyty w rakiecie FOK 2 jest w stanie wygenerować moment obrotowy na poziomie 0.1667 N m . Biorąc pod uwagę przełożenie w układzie wykonawczym systemu sterowania rakiety FOK 2, które wynosi $i_w = 2.75$, maksymalny moment wywierany na serwomechanizm przez ster wynosi 0.05543 N m . Do symulacji obciążenia aerodynamicznego steru użyto sprężyn o stałej sprężystości $k = 0.2000 \text{ N mm}^{-1}$. Umieszczono je na ramieniu $l = 25.00 \text{ mm}$ od osi obrotu steru. Jako, że zamocowane do steru są 2 sprężyny, to zakładając ich prostopadłość do ramienia steru, maksymalny moment wywierany na ster przez sprężyny wynosi $L = 2 \cdot k \cdot l^2 \cdot \sin \delta_{\max} = 0.04341 \text{ N m}$ co odpowiada wartości momentu zawiasowego dla liczby Macha $Ma = 0.3735488109$. Na potrzeby przyszłych testów zostanie odpowiednio zwiększone ramię sprężyn, tak by możliwe było symulowanie momentu zawiasowego steru dla liczby Macha $Ma = 0.7$. Zdecydowano się na użycie mniejszego momentu w celu sprawdzenia działania systemu w warunkach dalszych od granicznych serwomechanizmów.

4.3. Oprogramowanie

Oprogramowanie *CASKETa*, podobnie jak oprogramowanie rakiety FOK 2, jest oparte na FreeRTOS i napisane w języku C++. Oba urządzenia są obsługiwane przez ten sam mikrokontroler - Raspberry Pi Pico 2 W. Użycie tej samej platformy i systemu operacyjnego pozwala na zminimalizowanie liczby koniecznych kompetencji do obsługi obu urządzeń. Oprogramowanie *CASKET* jest celowo zaprojektowane tak, by było możliwie proste. Podstawowe funkcjonalności to:

1. odczyt danych z czujników,
2. przesyłanie danych do komputera osobistego lub komputera czasu rzeczywistego.

Ponad te funkcjonalności, oprogramowanie *CASKETa* umożliwia:

- obsługę podstawowych komend konfiguracyjnych,
- wyzerowanie czujników.

Podobnie jak w przypadku rakiety FOK 2, *CASKET* nie przechowuje trwale swoich stanów. W związku z tym, po każdym włączeniu, oprogramowanie *CASKETa* wymaga ponownej konfiguracji. Domyślna konfiguracja może być zmieniona przez rekompilację kodu źródłowego - jest zawarta w pliku `casketConfig.hpp`. Takie rozwiązanie eliminuje konieczność implementowania w oprogramowaniu *CASKETa* mechanizmów przechowywania konfiguracji, jednak wskazana jest możliwość konfigurowania *CASKETa* przez pliki konfiguracyjne, np. *YAML*, co pozwoliłoby osobom nieposiadającym kodu źródłowego lub skonfigurowanego środowiska programistycznego na zmianę konfiguracji *CASKETa*. Oprogramowanie *CASKETa* wykorzystuje oba rdzenie mikrokontrolera, jeden rdzeń odpowiedzialny jest za odczyt danych z czujników, a drugi za pozostałe funkcjonalności. Takie rozwiązanie pozwala na maksymalizację częstotliwości odczytu danych z czujników. Oprogramowanie *CASKETa* składa się z następujących *tasków*:

- odczyt danych z czujników,
- przesyłanie danych za pomocą SPI,
- przesyłanie danych za pomocą USB,
- przesyłanie danych za pomocą WiFi,
- obsługa komend konfiguracyjnych,
- miganie diodą LED sygnalizującą stan oprogramowania.

Ponadto task odczytujący dane z czujników na początku działania konfiguruje czujnik, tak by zapewnić pożądaną tryb pracy.

Struktura pakietu danych wysyłanego przez *CASKET*a jest następująca:

- 1 x uint64_t - timestamp w mikrosekundach,
- 4 x float - odczyt kąta obrotu z czujników w radianach,
- 4 x float - odczyt prędkości obrotu z czujników w radianach na sekundę.
- 1 x 0xFA,
- checksum - suma kontrolna pakietu danych - XOR całego pakietu.

Enkodowanie jest w kolejności big-endian.

4.4. Użytkowanie

Użytkowanie stanowiska *CASKET*a można podzielić na 3 etapy: przygotowanie stanowiska, umieszczenie rakiety w stanowisku i przeprowadzenie testu.

4.5. Przygotowanie stanowiska

W celu przygotowania stanowiska do testu należy sprawdzić sztywność połączeń profili aluminiowych oraz sprawdzić czy stoliki XY są w pobliżu położenia neutralnego lub w pozycji w jakiej były gdy przeprowadzono poprzedni test.

4.6. Umieszczenie rakiety w stanowisku

W *CASKET*cie umieszcza się system sterowania rakiety, opcjonalnie z modułem awioniki (Rysunek 4.3). W tym celu należy:

1. Wyjąć jeden z druków pozycjonujących raketę w stanowisku (na Rysunku 4.3 jest to druk różowy).
2. Zapewnić, że co najmniej jedna ze śrub mocujących insert górny systemu sterowania do korpusu rakiety jest odkręcona, pozwoli to na przytwierdzenie systemu sterowania do stanowiska.
3. Ostrożnie włożyć raketę w stanowisko, uważając by stery nie zahaczyły ani o profile (wymaga to przekręcenia raket w osi podłużnej) ani o czujniki. Jeżeli zajdzie taka konieczność to dostosować położenie czujników wykorzystując stoliki XY lub luzując profile aluminiowe na których stoliki są zamocowane i odsuwając je od rakiety.
4. Wkręcić śruby mocujące raketę do drukowanego adaptera. Sugeruje się użycie co najmniej 2 śrub mocujących, by zapewnić stabilność rakiety w stanowisku. Zaobserwowano przemieszczanie się rakiety względem stanowiska podczas testów przy braku śrub mocujących.

5. Włożyć drugi druk pozycjonujący rakietę w stanowisku.
6. Spozycjonować magnesy względem czujników. W tym celu należy użyć stolików XY do ustawienia czujników w osi magnesów. Należy tak umieścić magnesy by imitatory sterów znalazły się w drukach pozycjonujących, jednak tak by osie sterów nie były przez nie odchylane.

W pozycji testowej czujniki powinny znajdować się około 1.200 mm od magnesów [10]. W momencie tworzenia pracy jest to ustalane bez wykorzystania przyrządów, jednak planowane jest wytworzenie ustalającego przyrządu.

Ponadto, jeżeli używa się *skalibrowanych* serwomechanizmów, czyli takich, których wychylenie steru zostało zbadane i odpowiednie poprawki wprowadzone do komputera pokładowego, rakiety to można skalibrować CASKETa używając wychylenia sterów. W tym celu należy:

1. Wyzerować czujniki CASKETa w pozycji neutralnej sterów.
2. Wychylić stery do pozycji maksymalnej, sprawdzić odczyt.
3. Wychylić stery do pozycji minimalnej, sprawdzić odczyt.
4. Przenieść stół XY i powtórzyć procedurę.

Powyższe kroki należy powtarzać do momentu uzyskania satysfakcjonujących wyników. Ulepszenie wymienionej procedury opisane jest w podrozdziale 7.1.

4.7. Przeprowadzenie testu

Przed przeprowadzeniem testu należy sprawdzić konfigurację CASKETa. Zaleca się rekompilację i wgranie oprogramowania CASKETa do mikrokontrolera przed każdą serią testów.

W pliku konfiguracyjnym `casketConfig.hpp` można ustawić:

- przypisanie sterów do czujników,
- częstotliwość odczytu danych z czujników,
- częstotliwość przesyłania danych do komputera osobistego lub komputera czasu rzeczywistego,
- domyślny odbiornik danych - komputer osobisty lub komputer czasu rzeczywistego (można przełączyć w trakcie testu),
- konfigurację interfejsu SPI - piny, baudrate, długość słowa,
- konfigurację interfejsu UDP - port, adres IP.

Ponadto możliwe jest wprowadzenie następujących komend konfiguracyjnych w trakcie testu:

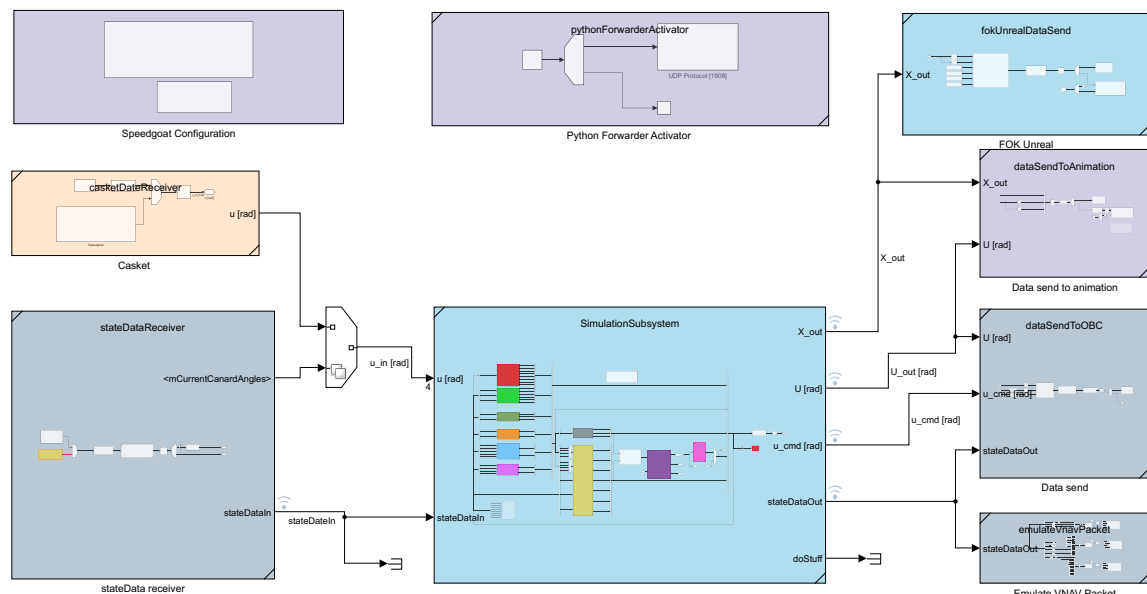
- `calibrate` - przeprowadza automatyczną kalibrację czujników opisaną w podrozdziale 7.1,
- `getstate` - wyświetla obecne pomiary wychylenia sterów i prędkości ich wychylenia w konsoli mikrokontrolera,
- `help` - wyświetla listę dostępnych komend,
- `reboot` - restartuje mikrokontroler,

4. Casket

- `serial` - ustawia wysyłanie danych do komputera osobistego po USB, argumenty to: `enable` lub `disable`,
- `spi` - ustawia wysyłanie danych do komputera czasu rzeczywistego po SPI, argumenty to: `enable` lub `disable`,
- `wifi` - ustawia wysyłanie danych do komputera osobistego po WiFi, argumenty to: `enable` lub `disable`.
- `zero` - ustawia zero czujników w aktualnej pozycji sterów.

5. Symulacja HIL

Trzon symulacji HIL stanowi symulacja rakiety sterowanej FOK 2 w Simulinku stworzona przez dra inż. Dariusza Miedzińskiego [11]. Została ona następnie przez Zespół rozwinięta. Dodano do niej między innymi łatwą możliwość konfiguracji rakiet i przetwarzanie danych. Przez autora pracy została zaadaptowana do służenia przeprowadzaniu symulacji HIL.



Rysunek 5.1. Widok główny symulacji HIL rakiety FOK 2

W rozdziale zawarty jest instruktaż jak uruchomić symulację HIL zarówno na komputerze osobistym jak i komputerze czasu rzeczywistego *Speedgoat*. Nie jest zagwarantowane, że instrukcje będą działać w przyszłości, jednak czytelnik powinien być w stanie ją zaadaptować do swoich potrzeb by przeprowadzić symulację nawet jeżeli w przyszłości zmieni się konfiguracja symulacji.

5.1. Dostosowanie symulacji do badań HIL

Stworzona symulacja HIL jest modułowa. W szczególności każdy z modułów jest w niej umieszczony za pomocą *Referenced Subsystem*. Pozwala to na pracę nad symulacją w wiele osób a także na wielokrotne używanie tych samych modułów jeżeli jest taka potrzeba. Największym zidentyfikowanym minusem takiego podejścia jest łatwość, z jaką można w modelu użyć złej wersji danego podsystemu. Może się tak zdarzyć przykładowo, gdy

mamy w ścieżce *MATLABa* kilka wersji symulacji. Dlatego przed właściwą symulacją rekomenduje się restart *MATLABa* lub ręczne przejrzanie zawartości ścieżki komendą `path`.

5.1.1. Komputer osobisty

Przeprowadzanie symulacji HIL na komputerze osobistym z użyciem *Simulinka* jest wykonywane za pomocą pakietu *Desktop Real-Time* [12]. Największym problemem w przypadku przeprowadzania symulacji na komputerze osobistym jest trudność w niezawodnej i pozbawionej opóźnień komunikacji z testowanym urządzeniem. Pakiet *Desktop Real-Time* nie udostępnia wielu możliwości komunikacji. Spośród dostępnych zdecydowano się wybrać *UDP protocol*. Jest to zestaw bloków umożliwiający komunikację w protokole UDP. Zaletą tego wyboru jest łatwość użycia - obok TCP jest to jeden z dwóch głównych protokołów warstwy transportowej infrastruktury sieciowej. Jest więc bez żadnych modyfikacji obsługiwany przez każdy komputer osobisty i wiele mikrokontrolerów. W praktyce, do jego użycia wystarczy jedynie podpiąć komputer przewodem ethernet lub bezprzewodowo do routera lub innego urządzenia.

Jednak nawet takie rozwiązanie nie jest pozbawione problemów. Pierwszym potencjalnym problemem jest brak gwarancji dostarczenia pakietu danych. W praktyce jednak współczesne rozwiązania sieciowe są na tyle niezawodne a użyta infrastruktura testowana na tyle prosta pod tym względem, że ten problem nie występuje. Większym problemem jest brak kontroli nad kolejką pakietów przychodzących do symulacji. Jako, że moduły systemu testowego mają być z założenia niezależne od siebie, to *CASKET* i komputer pokładowy zaczynają nadawać pakiety niezależnie od momentu rozpoczęcia symulacji. Zdiagnozowano, że takie zachowanie powoduje nabudowanie kolejki pakietów po stronie odbiorczej symulacji. Nie znaleziono prostego sposobu by czytać jedynie najnowszy pakiet z poziomu *Simulinka*. Stworzono więc obejście problemu. Urządzenia nie wysyłają pakietów bezpośrednio do symulacji, a do skryptu pythonowego. Zadaniem tego skryptu jest następnie przesłanie najnowszego pakietu do symulacji gdy symulacja zostanie uruchomiona. Zasada działania skryptu jest następująca:

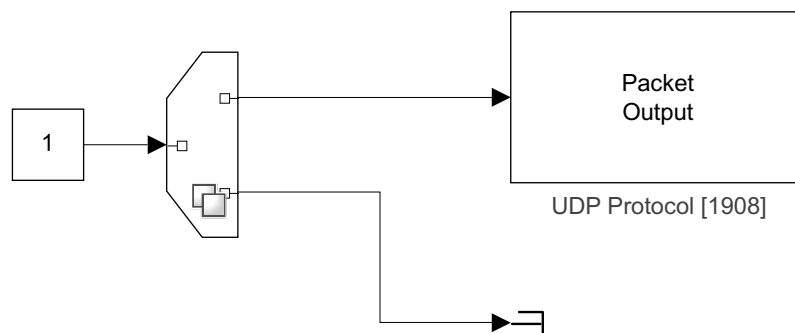
1. Oczekiwanie na pakiet aktywacyjny od symulacji.
2. Odebranie pakietu aktywacyjnego.
3. Przekazywanie w pętli najnowszych pakietów do symulacji.

Aktywator skryptu pythonowego polega na wysłaniu niepustego pakietu na odpowiedni port. Widoczny jest on na Rysunku 5.2

Takie rozwiązanie może potencjalnie zwiększać opóźnienia i zmniejszać deterministyczność komunikacji. W toku przeprowadzania testów nie zauważono jednak problemów wynikających z takiego przesyłania pakietów.

Warto nadmienić, że w toku projektu podjęto próbę unifikacji działania *CASKETa* w przypadku przeprowadzania symulacji na komputerze osobistym i czasu rzeczywistego. Użycie protokołu SPI do komunikacji z komputerem osobistym wymaga jednak użycia konwertera sygnału. Nie udało się znaleźć rozwiązania, które byłoby zarówno proste w konfiguracji, użyciu i działałoby z *Simulinkiem*.

W celu przeprowadzenia symulacji należy mieć zainstalowany pakiet *Desktop Real-Time*.



Rysunek 5.2. Moduł odpowiedzialny za aktywowanie skryptu przekierowującego pakiety do symulacji

Model został stworzony w wersji *Matlaba R2024b*. Był testowany jedynie na systemie operacyjnym Windows. Należy także posiadać kompilator C++ kompatybilny z tą wersją *Matlaba*.

W celu uruchomienia symulacji należy otworzyć skrypt inicjalizujący znajdujący się na repozytorium `fok-hil` w katalogu `Tests\PCSimulation` i otworzyć symulację `PCSimulation`. Należy ponadto uruchomić skrypt `pythonForwarder.py` znajdujący się na repozytorium `utils` w celu uruchomienia skryptu przekierowującego pakiety do symulacji. Należy upewnić poprawność przypisania portów w skrypcie i symulacji. W szczególności należy sprawdzić poprawność adresów IP w rakiecie, *CASKET*cie i symulacji. W celu uruchomienia symulacji należy przejść do zakładki *DESKTOP REAL-TIME*, wybrać *MODE* (w lewym górnym rogu) *Run in Kernel*. Następnie należy kliknąć *Run in Real Time*. Symulacja powinna się uruchomić.

5.1.2. Komputer czasu rzeczywistego

Mimo, że przeprowadzanie symulacji HIL jest możliwe na komputerze rzeczywistym to wiąże się to z pewnymi ograniczeniami. W szczególności trudnościami z obsługą interfejsów oraz zagwarantowaniem czasu rzeczywistego. Oba aspekty są szczególnie ważne gdy wynik symulacji zależy na komunikacji z innymi urządzeniami.

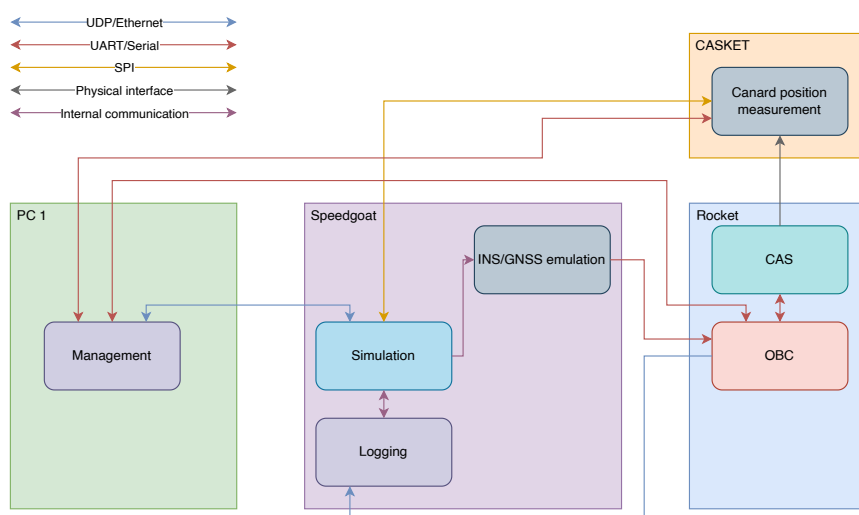
Przy komunikacji z *CASKET*em użycie go pozwoliło na bezpośrednie podpięcie mikrokontrolera obsługującego stanowisko pomiarowe do komputera czasu rzeczywistego używając protokołu SPI.

Drugim aspektem wykorzystania komputera czasu rzeczywistego do symulacji HIL jest wykorzystanie go do emulacji pakietów czujnika pokładowego rakiety - *VectorNav 200* [4]. W obecnej konfiguracji czujnik wysyła komputerowi pokładowemu 3 różne pakiety danych o częstotliwości odpowiednio 800.0 Hz, 400.0 Hz i 200.0 Hz:

1. przyspieszenie liniowe, prędkość kątowna,
2. dane z INS, parametry całkowania INS,

3. dane o niepewnościach, ciśnienie, odczyt magnetometru status czujników.

Ponadto każdy z pakietów zawiera timestamp. Jako, że interfejsem używanym w rakiemie do komunikacji OBC-czujnik jest UART (TTL), to możliwe jest bezpośrednia komunikacja *Speedgoat* - OBC, w której to *Speedgoat* emuluje czujnik. Teoretycznie z użyciem konwertera powinno być możliwe zaimplementowanie tej funkcjonalności w symulacji z komputerem osobistym jednak ta konfiguracja nie była dotąd rozwijana. Zaimplementowano jedynie komunikację jednostronną - *Speedgoat* emuluje już skonfigurowany czujnik bez możliwości rekonfiguracji przez komputer pokładowy jak ma to miejsce w przypadku konfiguracji lotnej. Takie rozwiązanie sprawia że z minimalnymi zmianami w konfiguracji komputera pokładowego możliwe jest przeprowadzenie symulacji, podczas której *sądzi on* że odbywa test lotny. Umożliwia to lepsze odwzorowanie działania komputera podczas lotu przez zapewnienie identycznych interfejsów a co za tym idzie interakcji między różnymi komponentami rakiety, czasami odczytu danych co w przypadku prawdziwego lotu.



Rysunek 5.3. Schemat konfiguracji CASKET-Speedgoat

Procedura przeprowadzania symulacji jest aktualna na dzień pisania pracy. Przeprowadzenie symulacji HIL na komputerze czasu rzeczywistego wymaga odpowiedniego skonfigurowania środowiska. Wymagane jest zainstalowanie *Simulink Real-Time* oraz *Speedgoat Target Support Package*. Ponadto wymagane jest użycie specjalnego zestawu bloków. Nie jest on publicznie dostępny, jednak członkowie Zespołu mają do nich dostęp. Na czas pisania pracy używana jest wersja *Matlaba R2024b*. Model był testowany jedynie na systemie operacyjnym Windows. Komputer czasu rzeczywistego *Speedgoat* jest w dyspozycji Zakładu Mechaniki, więc dostęp do niego możliwy jest dzięki uprzejmości pracowników Zakładu.

W celu przeprowadzenia symulacji należy podłączyć się do komputera czasu rzeczywistego *Speedgoat* za pomocą przewodu ethernet do portu *Host Link*. Sugerowana jest następująca kolejność działań:

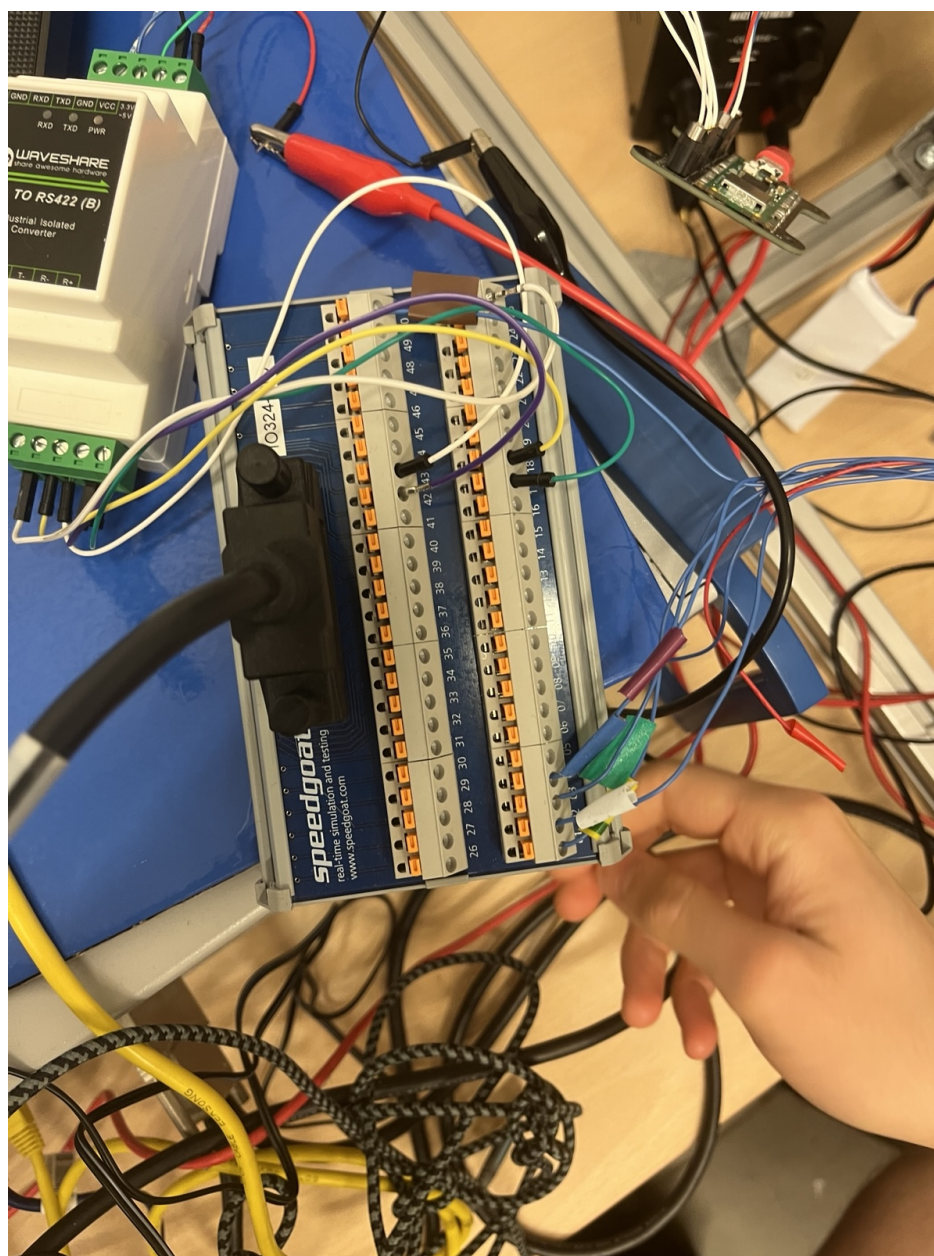
1. Uruchomienie *MATLABa*, załadowanie, otwarcie modelu symulacji.

2. Uruchomienie *Simulink Real-Time Explorer*. Wpisanie adresu IP komputera *Speedgoat* w polu *IP address*: 172.22.3.233.
3. Należy następnie nadać karcie sieciowej komputera osobistego adres statyczny adres IP: 172.22.3.69 i maske podsieci: 255.255.0.0.
4. Najłatwiej jest wyłączyć zaporę sieciową komputera osobistego dla sieci prywatnych na czas przeprowadzania symulacji. Należy pamiętać o jej ponownym włączeniu po zakończeniu symulacji.
5. Należy włączyć komputer czasu rzeczywistego *Speedgoat*.
6. Należy ustawić połączenie z komputerem czasu rzeczywistego jako sieć prywatną. Najłatwiej jest to zrobić uruchamiając *PowerShell* w trybie administratora i użyć następujących komend:
 - a) `Get-NetConnectionProfile` by wyświetlić listę połączeń sieciowych i znaleźć połączenie z komputerem *Speedgoat*.
 - b) `Set-NetConnectionProfile -InterfaceIndex N -NetworkCategory Private` gdzie N jest indeksem połączenia z komputerem *Speedgoat*. Ustawi to połączenie jako sieć prywatną.
7. Zweryfikować w ustawieniach sieciowych, że sieć jest ustawiona jako prywatna.
8. Zweryfikować, że zaporę sieciową jest wyłączona dla sieci prywatnych.
9. W *Simulink Real-Time Explorer* należy wybrać *TargetPC1* w lewym górnym rogu i kliknąć w *Disconnected* by połączyć się z komputerem *Speedgoat*.

Zdarza się, że połączenie z komputerem *Speedgoat* nie może zostać nawiązane. Nie znaleziono konkretnej przyczyny tego problemu. Po serii prób resetu komputerów, karty sieciowe, włączenia, wyłączenia zasilania został wyniesiony wniosek, że ten sam efekt można osiągnąć restartując sam komputer czasu rzeczywistego wielokrotnie, za każdym razem czekając po jego wyłączeniu i sprawdzając, czy możliwe jest połączenie po jego włączeniu. Zdarza się także, że połączenie zostaje nawiązane, jednak po próbie uruchomienia symulacji zostaje zerwane. Należy wtedy powtórzyć procedurę restartu komputera czasu rzeczywistego. Sugeruje się także rozłączyć z innych sieci, w których komputer osobisty może być podłączony, w szczególności z sieci Politechniki, ponieważ używa podobnych zakresów adresów IP.

Schemat portów interfejsu komputera czasu rzeczywistego *Speedgoat* jest dostępny po kliknięciu na blozki komunikacji w modelu symulacji. By połączyć *CASKET* z komputerem czasu rzeczywistego należy użyć portu SPI. Przykładowe połączenie przewodów do komputera czasu rzeczywistego *Speedgoat* jest przedstawione na Rysunku 5.4. *CASKET* jest podłączony do wyjść 01-04 i 50. Do portów 17-18, 42-43 i 25 podpięto przewody służące do emulacji czujnika INS/GNSS.

Na dzień pisania pracy skrypt ładujący model symulacji i sam model znajdują się na repozytorium fok-hil w folderze `Tests\SpeedgoatSimulation`. Należy uruchomić skrypt `initSpeedgoatSimulation.m` by załadować model symulacji by załadować parametry a następnie uruchomić model `SpeedgoatSimulation.slx`. Skrypt zmienia folder roboczy więc należy pamiętać, by nie wrócić podczas uruchamiania modelu do folderu z modelem.



Rysunek 5.4. Podłączenie przewodów do komputera czasu rzeczywistego *Speedgoat*

O ile możliwe jest użycie przycisku *Run on Target* do uruchomienia symulacji, to w celu umożliwienia szybszych restartów symulacji sugeruje się następującą procedurę:

1. Wybranie z listy rozwijanej pod przyciskami *Run on Target* opcji *Build Application*.
2. Kliknięcie w przycisk *Deploy to Target*.
3. Kliknięcie w przycisk *Connect Model*.
4. W *Simulink Real-Time Explorer* należy kliknąć w przycisk *Load Application*.
5. Wybrać *SpeedgoatSimulation*.
6. Kliknąć w przycisk *Start*.

Uruchomienie symulacji kolejny raz wymaga jedynie kliknięcia w przycisk *Start* po zakończeniu poprzedniej symulacji. Między poszczególnymi krokami należy poczekać, na zakończenie poprzedniego. Możliwe, że dany krok będzie trwał nawet kilka minut. Czas ten jest krótszy w przypadku mniej skomplikowanych modeli. Sugeruje się także uruchomienie *Data Inspector* w *Simulink Real-Time Explorer*. Po pierwszym uruchomieniu symulacji pojawią się tam dane wynikowe z symulacji i możliwe będzie stworzenie wykresów. Wraz z uruchamianiem kolejnych symulacji wykresy będą umożliwiały podglądanie wyników na żywo. Ponadto *Data Inspector* umożliwia zapisanie danych do pliku `.mat` w celu dalszej analizy.

6. Badania

W ramach pracy przeprowadzono 2 badania: identyfikację modelu układu wykonawczego oraz porównanie modelu z symulacją HIL na przykładzie misji pionizacji rakiety FOK 2.

6.1. Identyfikacja modelu

Na podstawie instrukcji [13] założono, że serwo generuje trapezowy profil prędkości [14]. W konfiguracji serwa ustawiono pożądany czas przyspieszania na 0 s tym samym sprawiając, że trajektoria trapezowa staje się trajektorią trójkątną. Część układu wykonawczego śledzącą wygenerowaną trajektorię zamodelowano jako układ drugiego rzędu. Ponadto na podstawie instrukcji [13] przyjęto, że serwomechanizm działa w krokach czasowych równych $h = 11.20 \text{ ms}$. Zakładając, że zadany kąt steru wynosi θ_f a początkowy przez θ_0 . Oznaczmy przez k dany krok czasowy serwa. Pomijając opóźnienie mamy wtedy zmianę kąta steru:

$$\theta_k = \theta_f - \theta_0 \quad (6.1)$$

Pożądaną prędkość serwa w danym kroku czasowym k można wyznaczyć ze wzoru:

$$\omega_k = \text{sgn} \theta_k \min \left(\omega_{\max}, \sqrt{2\epsilon_{\max} |\theta_k|} \right) \quad (6.2)$$

Gdzie na podstawie instrukcji [13] przyjęto, $\omega_{\max} = 4.451 \text{ rad s}^{-1}$. Wtedy po saturacji mamy:

$$\hat{\omega}_{k+1} = \omega_k + \text{sat}(\omega_k - \omega_k, \epsilon_{\max} h) \quad (6.3)$$

Ograniczając pożądaną prędkość serwa do wartości, która nie spowoduje przekroczenia zadanej pozycji steru w kroku czasowym $k + 1$ otrzymujemy:

$$\omega_{k+1} = \begin{cases} \frac{\theta_k}{h}, & |\hat{\omega}_{k+1}| h > |\theta_k| \\ \hat{\omega}_{k+1}, & \text{w przeciwnym wypadku} \end{cases} \quad (6.4)$$

Pożądaną pozycję steru w kroku czasowym $k + 1$ można wtedy wyznaczyć ze wzoru:

$$\theta_{k+1} = \theta_k + \omega_{k+1} h \quad (6.5)$$

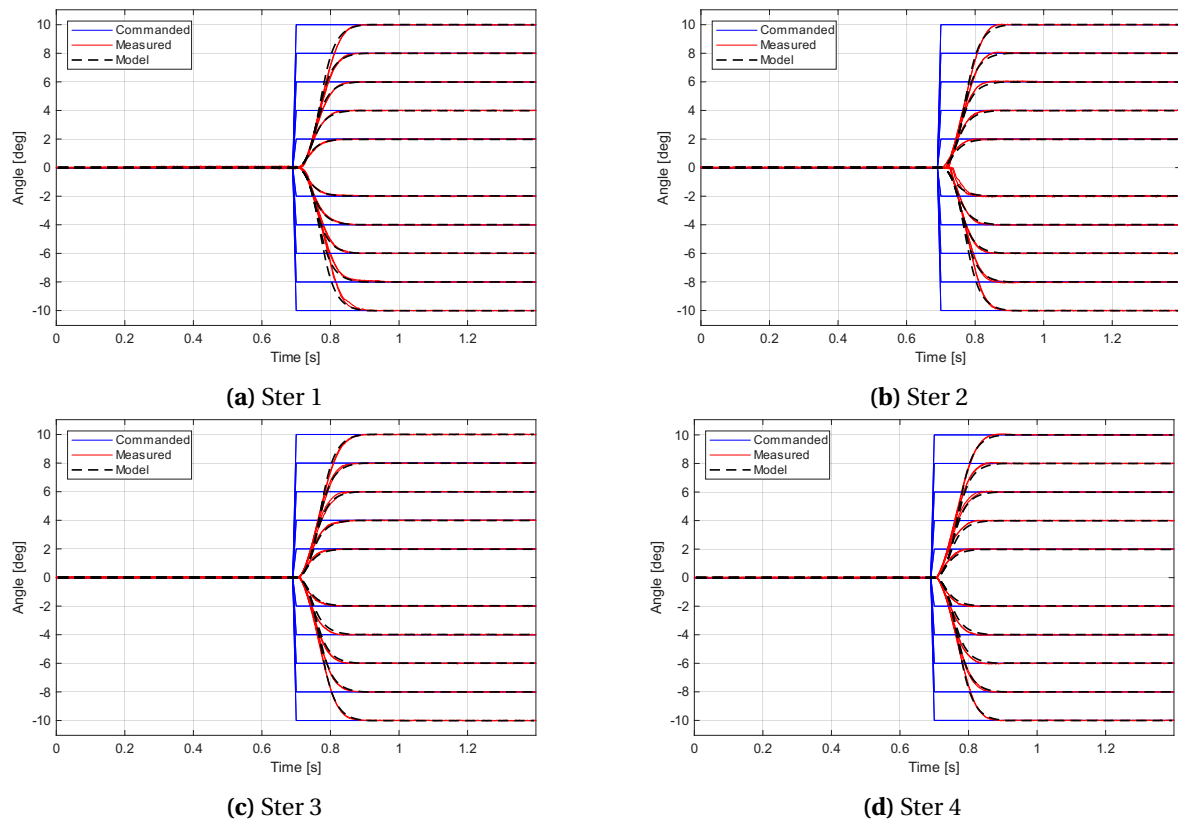
Wtedy model drugiego rzędu śledzący pożądaną pozycję steru można opisać równaniem:

$$\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = \omega_n^2\theta_{k+1} \quad (6.6)$$

Przy czym pozycja θ i pożądana pozycja θ_{k+1} są skwantyzowane zgodnie z dokładnością enkodera serwa: 0.3250° dla serwa więc w przypadku rakiety FOK 2D 0.1182° dla steru.

W celu dobrania parametrów ω_n , ζ oraz $\epsilon_{\max}$ przeprowadzono eksperymenty polegające na zadawaniu wychylenia skokiem o amplitudach w zakresie $\pm 10.00^\circ$ co 2.000° . Następnie przeprowadzono optymalizację na wszystkich sygnałach wyjścia jednocześnie.

Takie podejście naturalnie uśrednia parametry modelu dla wszystkich serw, podczas gdy rzeczywiste stery mogą się od siebie różnić. Wyniki procesu optymalizacji przedstawiono na Rysunku 6.1. Dobrane parametry modelu umieszczono w Tabeli 6.1. Natomiast na Rysunku 6.2 przedstawiono porównanie odpowiedzi modelu z rzeczywistą odpowiedzią serw na sygnał wieloczęstotliwościowy.

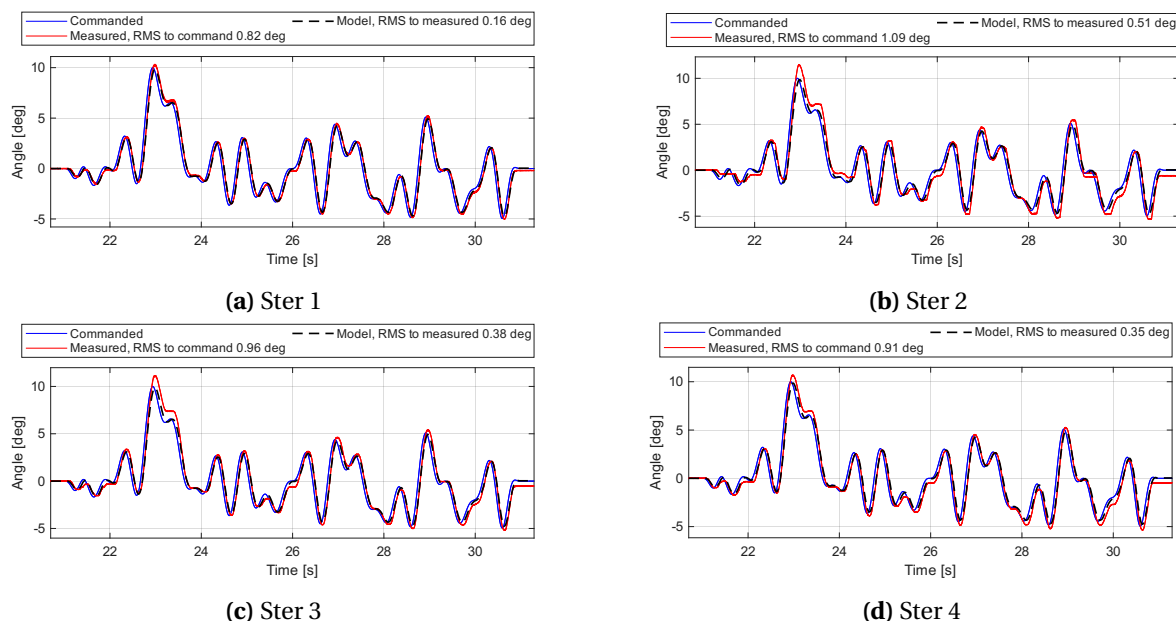


Rysunek 6.1. Porównanie odpowiedzi modelu z rzeczywistą odpowiedzią serw na sygnał skokowy

Tabela 6.1. Dobrane parametry modelu serw

Parametr	Wartość	Jednostka
ω_n	53.23	rad s^{-1}
ζ	1.016	-
$\epsilon_{\max}$	59.38	rad s^{-2}

Oraz przyjęto opóźnienie komunikacyjne $\tau = 5.000 \text{ ms}$ co odpowiada jednemu krokowi czasowemu komputera pokładowego rakiety FOK 2D.



Rysunek 6.2. Porównanie odpowiedzi modelu z rzeczywistą odpowiedzią serw na sygnał wieloczęstotliwościowy

Na Rysunku 6.1 widać, że odpowiedzi niektórych serw nieco odbiega od innych (przykładowo ster 2 (Rysunek 6.1b)). Podczas badań zaobserwowano też, nietypowe zachowanie serwomechanizmów w niektórych przebiegach. Może być to spowodowane stanem rozładowania akumulatora. Sugeruje się więc wykonanie większej liczby badań.

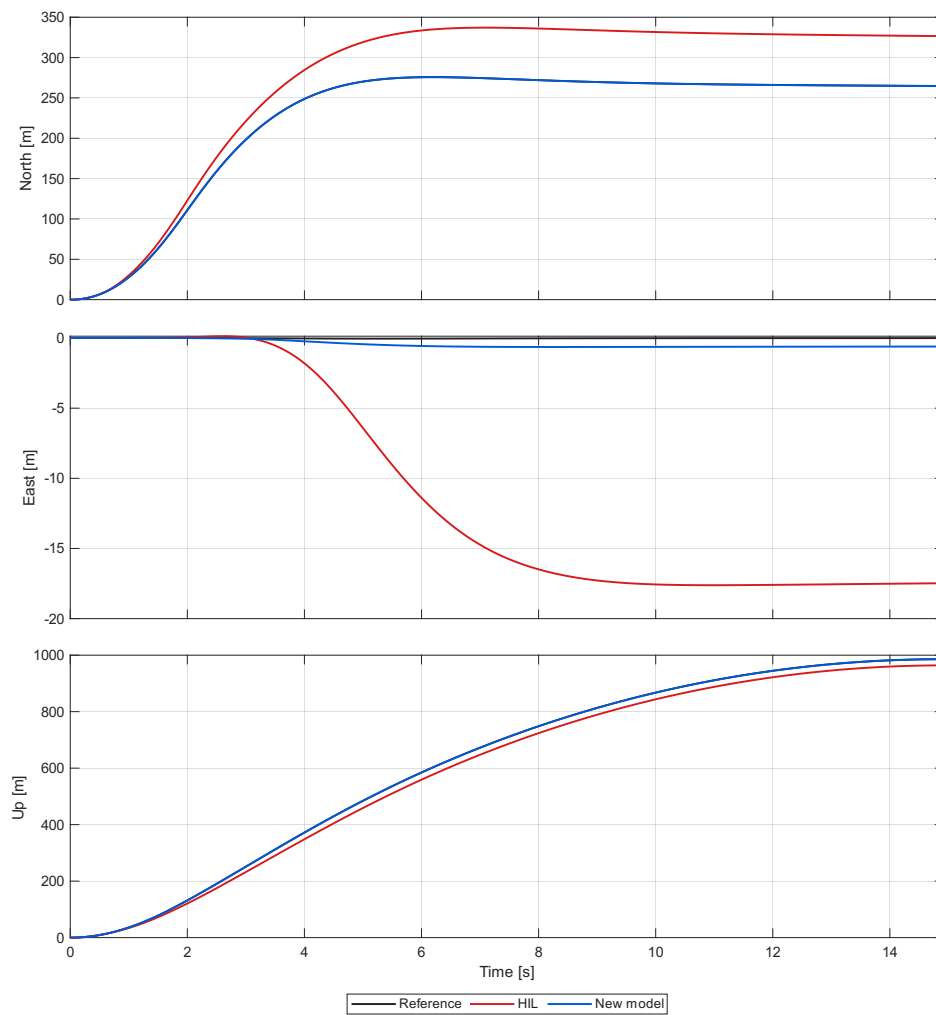
6.2. Symulacja HIL

Używając modelu systemu sterowania wyznaczonego w podrozdziale 6.1 przeprowadzono porównanie symulacji HIL i MIL na przykładzie misji pionizacji rakiety FOK 2D. Rakieta zaczynała z elewacji $\theta = 55.00^\circ$ i azymutu $\phi = 0^\circ$ i miała osiągnąć pozycję pionową, zaczynając sterowanie 1.700 s po wykryciu startu. Opóźnienie względem tego czasu wynika z filtra dolnoprzepustowego używanego w algorytmie wykrywania startu rakiety. Przeprowadzono 3 symulacje: symulację używając starego modelu systemu sterowania wyznaczonego w ramach [5], pierwszego rzędu, oznaczoną jako *Reference*. Modelu drugiego rzędu wyznaczonego w ramach podrozdziału 6.1, oznaczoną jako *New model* oraz symulację HIL z użyciem rzeczywistego CAS, oznaczoną jako *HIL*.

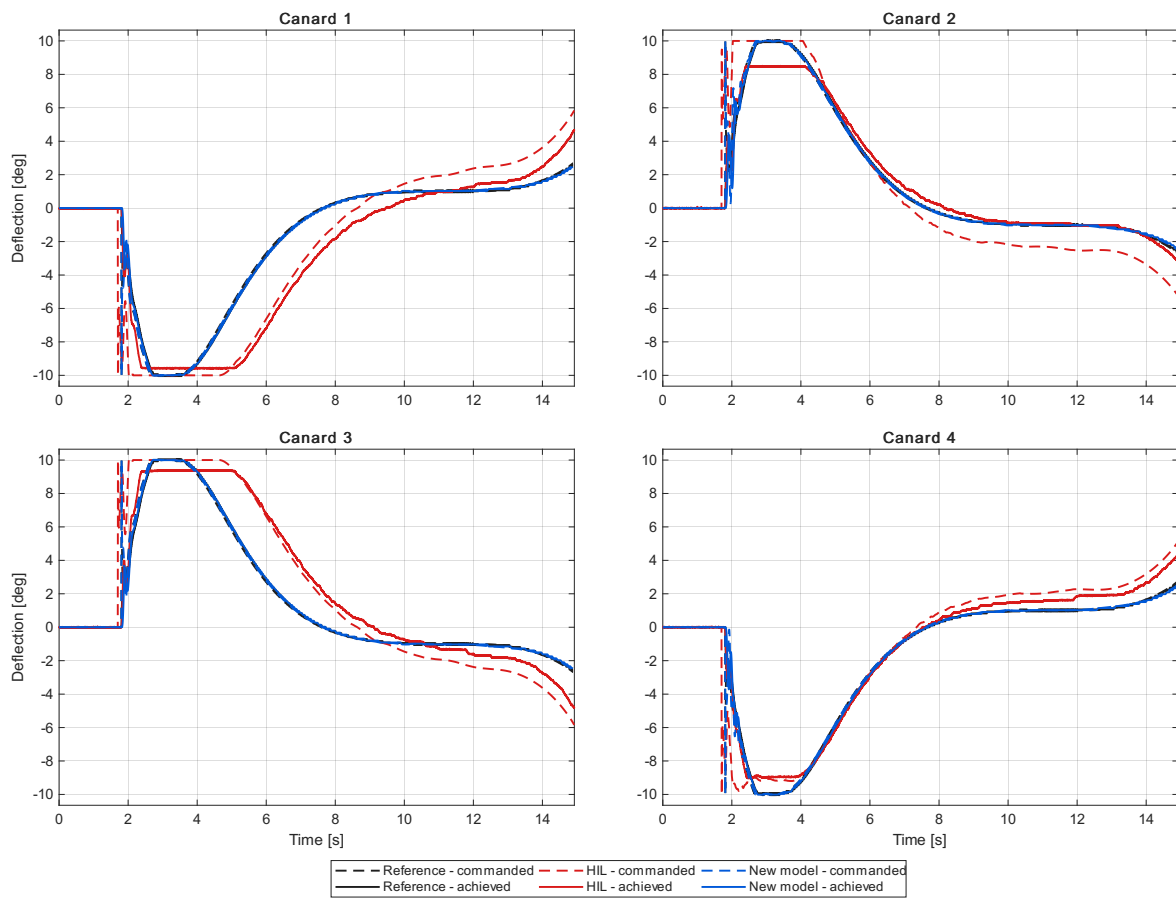
Obserwując trajektorię rakiety przedstawioną na Rysunku 6.3 widać, że w każdym z przypadków rakieta osiąga pozycję pionową. Różnice między obiema symulacjami MIL są niewielkie. Natomiast w przypadku symulacji HIL widać znacznie późniejsze osiągnięcie pozycji pionowej. Najprawdopodobniej spowodowane jest to przez brak osiągnięcia przez serwa pożądanej pozycji 10.00° (Rysunek 6.4). Jest to spowodowane opisanym w [1], [5] problemem z dokładnością enkodera serw. Przedstawiona symulacja została przeprowadzona bez kompensacji błędu enkodera.

Należy także zauważyć, że opóźnienie w zidentyfikowanym modelu drugiego rzędu jest większe niż w przypadku rzeczywistego systemu sterowania. Może to wynikać z faktu, że *tick* serwa w rzeczywistym teście wypadł w innym momencie symulacji lub innym

zachowaniem serwa niż zamodelowane. Rozbieżność wskazuje na konieczność przeprowadzenia większej liczby badań w celu wyznaczenia bardziej dokładnego modelu systemu sterowania rakiety FOK 2D.



Rysunek 6.3. Porównanie trajektorii pionizacji rakiety FOK 2D w symulacji HIL i MIL



Rysunek 6.4. Porównanie wychyleń sterów rakiety FOK 2D w symulacji HIL i MIL

7. Rozwój

W toku dotychczasowych prac nad stanowiskiem testowym [1], [7] zidentyfikowano kilka aspektów do poprawy w działaniu stanowiska, które nie zostały zrealizowane w ramach tej pracy. Te aspekty to:

- Możliwość wprowadzenia korekty mierzonych wychyleń sterów.
- Zwiększenie częstotliwości wysyłania danych przez *CASKET*.

7.1. Wbudowana w *CASKET* korekta wychyleń sterów

Obecnie użycie bezkontaktowych czujników magnetycznych sprawia, że pomiar kąta wykonywany przez *CASKET* jest niedokładny i zależy od względnego ustawienia czujnika i magnesu. Zmitygowanie tego problemu polega na iteracyjnych korektach ustawienia czujnika względem magnesu. Podczas projektowania *CASKETa* została przewidziana taka ewentualność - jest to jedna z przyczyn zastosowania stolików ze śrubami mikrometrycznymi. Proces ten, o ile skuteczny i jednorazowy dla jednego włożenia układu sterowania w urządzenie pomiarowe, o tyle znacznie wydłuża czas przygotowania do pierwszego testu, a jego wynik w dużej mierze zależy od operatora wprowadzającego poprawki. Pożądane jest więc wprowadzenie automatyzacji procesu. Korekcja względnego ustawienia magnesu i czujnika wciąż jest wskazana, jednak powinna być wspomagana przez poprawki wprowadzone w oprogramowaniu stanowiska. Zostanie stworzony skrypt, którego zadaniem będzie automatyczne wygenerowanie poprawek pomiarowych dla *CASKETa*. Jego działanie będzie polegało na wychyleniu sterów na trzy pozycje: 2 skrajne i zerową, sczytanie pomiarów *CASKETa* i następnym takim skorygowaniu pomiarów *CASKETa*, by te punkty leżały możliwie blisko prostej $y = x$. Planuje się do tego zastosować przekształcenie liniowe. Należy pamiętać o wpływie luzów występujących w systemie. Każdy z punktów zostanie osiągnięty w dwóch kierunkach - raz ster się wychyli do położenia 2.000° większego, raz o 2.000° mniejszego. Wartość pomiaru zostanie więc uśredniona z dwóch kierunków. Tym samym zostanie zmitygowany wpływ luzów w systemie na pomiar wychylenia steru. Kalibrację należy wykonywać na systemie nieobciążonym, tj. pozbawionym sprężyn. Ta automatyczna procedura może też stanowić ulepszenie względem obecnego ustawiania zera czujników, mitygując problem luzów w występujący w obecnie używanej procedurze.

7.2. Zwiększenie częstotliwości *CASKETa*

O ile możliwe jest prowadzenie pomiarów z częstotliwością 1600 Hz, to w toku testów zdiagnozowano czasową utratę sygnału z *CASKETa*. Problem rozwiązano zmniejszając częstotliwość wysyłanego wychylenia do 800.0 Hz. Jest to wciąż duża częstotliwość i zmniejszenie jej nie wpływa znacząco na symulację. Obecnie używanym baudrate jest $2.000 \times 10^6 \text{ bits s}^{-1}$, został on uznany za niezawodny eksperymentalnie. Przy tym baudrate odczytanie wychylenia sterów z *CASKETa* powinno zająć 64.00 μs . Natomiast wysłanie pakietu z wychyleniem sterów zajmuje 168.0 μs . Działanie systemu z częstotliwością 1600 Hz

powinno być więc teoretycznie możliwe w wyniku lepszego zoptymalizowania oprogramowania *CASKETa*.

Zwiększenie częstotliwości wysyłanych danych możliwe będzie gdy użyte zostanie *Direct Memory Access* (DMA) na liniach komunikacyjnych. Pierwotnie oszacowano, że zaimplementowanie tej funkcjonalności nie będzie konieczne, jednak przeprowadzone testy wykazały, że była to fałszywa hipoteza.

8. Podsumowanie

W ramach pracy stworzono stanowisko testowe do przeprowadzania badań HIL układu wykonawczego systemu sterowania rakiety sterowanej FOK 2. Przeprowadzono wstępne badania używając stanowiska testowego, tym samym potwierdzając jego poprawne działanie i użyteczność w badaniach HIL. Przeprowadzone badania uwidoczniły też kilka aspektów, które wymagają poprawy w działaniu stanowiska. W szczególności zidentyfikowano potrzebę wprowadzenia korekty pomiaru wychyleń sterów w oprogramowaniu stanowiska oraz uwidoczniło wrażliwość stanowiska na względne ustawienie czujnika i magnesu. Zidentyfikowano obszary, w których należy w najbliższym czasie ulepszyć stanowisko testowe.

Ponadto opisano proces przeprowadzania symulacji HIL na komputerze osobistym z użyciem *Simulinka* oraz na komputerze czasu rzeczywistego *Speedgoat*. Pozwoli to członkom Zespołu na samodzielne przeprowadzanie symulacji HIL w przyszłości i ułatwi proces przekazywania wiedzy o działaniu symulacji HIL.

Stworzone stanowisko testowe jest integralną częścią stanowiska testowego całej rakiety FOK 2, które umożliwia przeprowadzanie badań HIL pętli sterowania zawierającej zarówno głowicę obrazującą, czujnik INS/GNSS, komputer pokładowy i układ wykonawczy systemu sterowania rakiety. Nabyte możliwości w zakresie przeprowadzania symulacji HIL pozwolą na lepszą weryfikację działania rakiety FOK 2 przed planowanymi testami lotnymi. W szczególności na sprawdzenie poprawności działania pętli sterowania rakiety, jej oprogramowania pokładowego oraz rozważenie scenariuszy awaryjnych, które mogą wystąpić podczas testów lotnych.

Bibliografia

- [1] O. Skromak, B. Dąbrowski, S. Dylewski i in., „Hardware-in-the-Loop testing solution for a guided rocket control actuation system”, w *Proceedings of the 27th ESA Symposium on European Rocket and Balloon Programmes and Related Research*, 2026.
- [2] M. Tomporowski, „Projekt wstępny silnika rakietowego typu ciąg-podtrzymanie”, Engineering Thesis, Warsaw University of Technology, 2025.
- [3] Amazon Web Services. „FreeRTOS”. Real-time operating system kernel for embedded devices. (), adr.: <https://www.freertos.org/> (term. wiz. 12. 09. 2026).
- [4] VectorNav Technologies. „VN-200 GNSS/INS”. Karta produktu. (), adr.: <https://www.vectornav.com/products/detail/vn-200> (term. wiz. 12. 09. 2026).
- [5] O. Skromak, „Study of the control actuation system of a rocket using Hardware-in-the-Loop method”, Engineering Thesis, Warsaw University of Technology, 2025.
- [6] O. Skromak, M. Mikulski, K. Zyśk i in., *Hardware-in-the-Loop Testing Solution for a Guided Rocket Vision System*, Presentation, Paper No. 132, 28th AIDAA International Congress / 10th CEAS Aerospace Europe Conference (CEAS-AIDAA 2025), Turin, Italy, grud. 2025.
- [7] O. Skromak, B. Dąbrowski, S. Dylewski i in., „Hardware-in-the-Loop testing solution for a guided rocket”, w *ESA GNC & ICATT Conference 2026*, (28 wrz.–2 paź. 2026), European Space Agency, Sevilla, Spain, 2026. adr.: <https://atpi.eventsair.com/gnccicatt/>.
- [8] Monolithic Power Systems, *MA600: High-Accuracy, <0.6° (<0.1°) INL, High-Bandwidth, Configurable Digital Magnetic Angle Sensor*, wer. Rev. 1.0, url: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MA600GQ/document_id/10535/, Monolithic Power Systems, Inc., 28 czer. 2023. (term. wiz. 17. 05. 2026).
- [9] M. Moraczewski, K. Bielaszka, J. Brzostek i in., „Aerodynamic Optimization of Canard Geometries for the FOK Guided Rocket”, w *Proceedings of the 27th ESA Symposium on European Rocket and Balloon Programmes and Related Research*, (Trondheim, Norway, 31 maj.–4 czer. 2026), European Space Agency (ESA), 2026. adr.: <https://www.esapac.no/event/07453893-53be-496c-ba8c-02e02bb974ab/detailed-schedule> (term. wiz. 12. 09. 2026).
- [10] Monolithic Power Systems, „Selecting the Right Magnet for the MagAlpha in End-of-Shaft Mounting”, Monolithic Power Systems, Application Note, sty. 2017, SKU: MA730GQ-Z, Document ID: 6303. adr.: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Application%20Note/lang/en/sku/MA730GQ-Z/document_id/6303/ (term. wiz. 12. 09. 2026).
- [11] D. Miedziński, *Analiza zachowania modeli liniowych i nieliniowych przy badaniu stateczności i sterowności rakiet*, 2024.
- [12] T. M. Inc., *Simulink Desktop Real-Time (R2024b)*, Natick, Massachusetts, United States, 2026. adr.: <https://www.mathworks.com/help/sldrt/>.

- [13] Dongbu Robot, *HerkuleX DRS-0101/DRS-0201 User Manual*.
- [14] L. Biagiotti i C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Berlin, Heidelberg: Springer, 2008, 514 s., ISBN: 978-3-540-85628-3. DOI: 10.1007/978-3-540-85629-0.

Wykaz symboli i skrótów

Skróty

CAS	— Control Actuation System
CASKET	— Control Actuation System Kinematics Examination Tool
DMA	— Direct Memory Access
HIL	— Hardware-in-the-Loop
MIL	— Model-in-the-Loop
OBC	— Onboard computer
SIL	— Software-in-the-Loop
SKA	— Studenckie Koło Astronautyczne
SPI	— Serial Peripheral Interface
UDP	— User Datagram Protocol

Symbole

ω_n	— częstotliwość drgań własnych
ζ	— tłumienie
$\epsilon_{\max}$	— maksymalne przyspieszenie kątowe

Spis rysunków

2.1	Rakieta FOK 2 w tunelu aerodynamicznym	6
2.2	Wymiary rakiety FOK 2	6
2.3	Pętla sterowania rakiety FOK 2	7
2.4	Układ wykonawczy systemu sterowania rakiety FOK 2	8
4.1	Casket connectivity diagram	13
4.2	Zamienniki sterów rakiety FOK 2 z magnesami i zaczepami do sprężyn	14
4.3	FOK 2 w stanowisku CASKET [1]	14
5.1	Widok główny symulacji HIL rakiety FOK 2	19
5.2	Moduł odpowiedzialny za aktywowanie skryptu przekierowującego pakiety do symulacji	21
5.3	Schemat konfiguracji CASKET-Speedgoat	22
5.4	Podłączenie przewodów do komputera czasu rzeczywistego <i>Speedgoat</i>	24
6.1	Porównanie odpowiedzi modelu z rzeczywistą odpowiedzią serw na sygnał skokowy	27
6.2	Porównanie odpowiedzi modelu z rzeczywistą odpowiedzią serw na sygnał wieloczęstotliwościowy	28
6.3	Porównanie trajektorii pionizacji rakiety FOK 2D w symulacji HIL i MIL	29

6.4	Porównanie wychyleń sterów rakiety FOK 2D w symulacji HIL i MIL	30
-----	---	----

Spis tabel

2.1	Parametry rakiety FOK 2.	7
6.1	Dobre parametry modelu serw	27